

BÁO CÁO KẾT QUẢ THỬ NGHIỆM

Sinh viên thực hiện: Võ Quốc Thịnh

Mã số sinh viên : 25521787

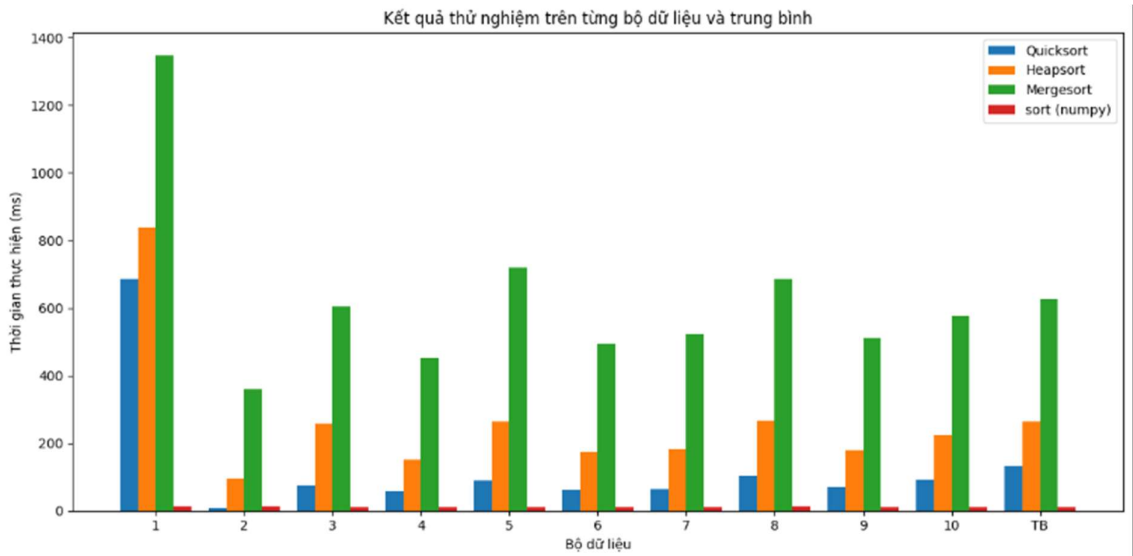
Nội dung báo cáo: Thực nghiệm các giải thuật sắp xếp nội

I. Kết quả thử nghiệm

1. Bảng thời gian thực hiện¹

Dữ liệu	Thời gian thực hiện (ms)			
	Quicksort	Heapsort	Mergesort	sort (numpy)
1	686.81520	838.17150	1347.80390	13.69980
2	8.97020	95.67190	360.58980	11.96890
3	76.57250	257.70750	604.09120	11.51570
4	57.07380	152.28690	453.48940	10.05740
5	89.11590	263.79540	720.24920	11.48510
6	60.58500	174.21330	495.40700	10.14530
7	63.45560	183.01320	522.47530	9.26640
8	102.24080	265.59570	686.34410	14.14150
9	68.19900	179.00320	511.90040	11.22450
10	90.94490	224.95840	575.49850	10.47580
Trung bình	130.3973	263.4417	627.7849	11.3980

2. Biểu đồ (cột) thời gian thực hiện



II. Kết luận:

¹ Số liệu chỉ mang tính minh họa

Dựa trên kết quả thực nghiệm thời gian thực hiện của các thuật toán sắp xếp trên 10 bộ dữ liệu khác nhau, có thể rút ra một số nhận xét và kết luận như sau:

Trước hết, **thuật toán sort (numpy)** cho hiệu năng vượt trội và ổn định nhất. Thời gian thực hiện của thuật toán này dao động rất nhỏ giữa các bộ dữ liệu và có giá trị trung bình chỉ khoảng **11.4 ms**, thấp hơn rất nhiều so với các thuật toán còn lại. Điều này cho thấy các hàm sắp xếp được tối ưu trong thư viện có ưu thế rõ rệt nhờ việc cài đặt ở mức thấp và áp dụng các kỹ thuật tối ưu hóa hiệu năng.

Tiếp theo, trong nhóm các thuật toán sắp xếp tự cài đặt, **Quicksort** cho kết quả tốt hơn so với **Heapsort** và **Mergesort** trên phần lớn các bộ dữ liệu. Tuy nhiên, thời gian thực hiện của Quicksort biến động khá lớn, đặc biệt ở **bộ dữ liệu 1**, cho thấy thuật toán này nhạy cảm với thứ tự dữ liệu đầu vào và có thể rơi vào trường hợp xấu.

Đối với **Heapsort**, mặc dù có độ phức tạp thời gian ổn định trong trường hợp xấu nhất, thời gian thực hiện trung bình vẫn khá cao (khoảng **263.4 ms**). Nguyên nhân chủ yếu là do chi phí thao tác trên cấu trúc heap trong quá trình sắp xếp.

Cuối cùng, **Mergesort** là thuật toán có thời gian thực hiện lớn nhất trong các thuật toán được khảo sát, với giá trị trung bình khoảng **627.8 ms**. Việc phải sử dụng thêm bộ nhớ phụ và các thao tác sao chép dữ liệu khiến hiệu năng của thuật toán này giảm đáng kể trong thực tế.

Tổng kết lại, kết quả thực nghiệm cho thấy việc sử dụng các hàm sắp xếp có sẵn trong thư viện là lựa chọn hiệu quả nhất về mặt hiệu năng. Trong trường hợp cần tự cài đặt thuật toán, **Quicksort** là lựa chọn phù hợp khi dữ liệu đầu vào không ở trạng thái bất lợi, trong khi **Heapsort** và **Mergesort** thích hợp hơn cho các bài toán có yêu cầu đặc thù về tính ổn định hoặc đảm bảo độ phức tạp trong trường hợp xấu nhất.

III. Thông tin chi tiết – link github, trong repo gibub cần có

1. Báo cáo
2. Mã nguồn
3. Dữ liệu thử nghiệm

Link repo : <https://github.com/thinhtheconqueror/Sorting-Algorithms>